COURSE CODE – R26XXXX	COMPUTATIONAL THINKING AND PROBLEM SOLVING USING C	ESC	L-T-P 3-0-0	CREDITS : 3

Pre-requisite: Mathematics and logical reasoning at 10+2 level. No prior programming experience is assumed.

Course Objectives: The main objectives of the course are to

- Familiarize students with computers, computational thinking, and the process of algorithmic problem solving.
- Build strong programming fundamentals using the C language, including operators, control structures, and loops.
- Develop the ability to design and implement solutions using arrays, strings, and user-defined data types.
- Impart working knowledge of pointers, recursion, dynamic memory allocation, functions, and file handling for building modular, reusable programs.

Course Outcomes: *At the end of the course, student will be able to*

		Knowledge Level (K)#
CO1	Apply computational thinking techniques (decomposition, abstraction, pattern recognition) to design algorithms and flowcharts for a given problem.	K3
CO2	Develop C programs using appropriate operators, expressions, selection statements, and loops to implement a given logic.	K3
CO3	Analyze problems involving arrays, strings, structures and unions and construct suitable C solutions.	K4
CO4	Apply pointers, recursion, and dynamic memory allocation to solve computational problems efficiently.	K3
CO5	Construct modular C programs using user-defined functions and perform text/binary file input-output operations.	K3

#Based on suggested Revised BTL

Mapping of course outcomes with program outcomes

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	H	H	M	L	-	-	-	-	-	-	L	H	-	-
CO2	H	M	H	-	M	-	-	-	-	-	-	H	-	-
CO3	H	H	H	M	-	-	-	-	-	-	L	H	-	-
CO4	H	H	H	M	M	-	-	-	-	-	-	H	-	-
CO5	H	M	H	L	M	-	-	-	L	-	L	H	-	-

(Please fill the above with Levels of Correlation, viz., L, M, H)

UNIT	CONTENTS	Contact Hours
UNIT – 1	**Computational Thinking and Introduction to C** **Computational Thinking:** Decomposition, Pattern Recognition, Abstraction, Algorithm Design; representing solutions as algorithms, pseudocode, and flowcharts; Number Systems and Data Representation (integers, reals); Overview of Computer Organization and Program Execution. **Introduction to C:** Background, Structure of a C Program, Identifiers, Keywords, Variables, Constants, Basic Data Types, Input/Output statements; Expressions, Operators, type of operators- arithmetic, relational, logical, bitwise, assignment, conditional, increment, decrement, etc., unary, binary, ternary, Operator Precedence and Associativity, Type Conversion; Scope, Storage Classes, and Type Qualifiers.	12
UNIT – 2	**Selection and Decision-Making**: Simple if, Two-Way selection- if-else, nested-if, else-if ladder, and Multi-Way Selection- switch. **Repetition:** Counter- and Event-Controlled Loops: for, while, do-while, and nested loops; Break and continue, Programming Applications combining Selection and Repetition. **Arrays:** One-Dimensional and Two-Dimensional Arrays, Multidimensional Arrays, Array-based Problem Solving (searching, sorting, etc.).	9
UNIT – 3	**Functions:** Designing Structured/Modular Programs, Function Declaration and Definition, Parameter Passing (by value, by reference using pointers), Passing Arrays and Pointers to Functions, Standard Library Functions, Inter-Function Communication. **Recursion:** Recursive vs. Iterative Solutions, Designing Recursive Algorithms, Recursion with Arrays and Pointers, Efficiency Considerations. **Strings:** String Representation, Standard String-Handling Functions, Arrays of Strings, String-to-Data Conversion.	9
UNIT – 4	**Pointers:** Pointer Fundamentals, Pointer-to-Pointer, Pointer Arithmetic, Pointers and Arrays, Array of Pointers, types of pointers, Compatibility and L-value/R-value concepts. Dynamic Memory Allocation: malloc, calloc, realloc, free. **User-Defined Types:** typedef, Enumerated Types, Structures, Unions, and Nested Structures; Programming Applications using Structures and Arrays of Structures.	9
UNIT – 5	**Pre-processor**: Directives and Macros. Command Line Arguments. **File I/O:** Streams, Text vs. Binary Files, Standard Library File Functions, formatted and Character I/O, Sequential File Processing.	6
	Total	45

Text Books:

1. Behrouz A. Forouzan & Richard F. Gilberg, Computer Science: A Structured Programming Approach Using C, 3rd Edition, Cengage Learning, ISBN-13: 978-0-534-49132-1.
2. Brian W. Kernighan & Dennis M. Ritchie, The C Programming Language, 2nd Edition, Pearson Education / Prentice Hall India, 1988.

Reference Books:

1. Sumitabha Das, "Computer Fundamentals and Programming", 18th Edition, McGraw Hill Education, 2018.
2. Ashok N. Kamthane, Amit Ashok Kamthane, "Programming in C", 4th Edition, Pearson Education, 2025. ISBN: 978-9332543633.
3. Pradip Dey, Manas Ghosh, "Computer Fundamentals and Programming in C", 2nd Edition, Oxford University Press, 2013. ISBN: 978-0198068842.

Web References:

1. NPTEL – Programming in C, IIT Kanpur: https://onlinecourses.nptel.ac.in/noc21_cs81/preview
2. Geeks for Geeks – C Programming Language: https://www.geeksforgeeks.org/c-programming-language/
3. Programiz – Learn C Programming: https://www.programiz.com/c-programming
4. cplusplus.com – C Standard Library Reference: https://cplusplus.com/reference/clibrary/
5. TutorialsPoint – C Programming: https://www.tutorialspoint.com/cprogramming/index.html
6. W3Schools – C Tutorial: https://www.w3schools.com/c/

COURSE CODE – R26XXXX	COMPUTATIONAL THINKING AND PROBLEM SOLVING USING C LAB	ESC	L-T-P 0-0-3	CREDITS : 1.5

Pre-requisite: - Basic understanding of computer fundamentals and problem-solving concepts

Course Objectives: The main objectives of the course are to

- Apply the principles of C language and computational thinking techniques in problem solving.
- Design flowcharts and algorithms, and to develop the ability to trace and debug programs.
- Design and develop C programs using arrays, strings, pointers, recursion, and functions.
- Apply dynamic memory allocation, file operations, and preprocessor commands in program design.

Course Outcomes: *At the end of the course, student will be able to*

CO	Course Outcome Statement	Knowledge Level (K)#
CO1	Apply computational thinking to construct algorithms and flowcharts for a given problem and translate them into working C programs.	K3
CO2	Develop C programs using operators, selection statements and loops to implement a specified logic.	K3
CO3	Develop C programs that manipulate arrays, strings, structures and pointers to solve computational problems.	K3
CO4	Construct modular programs using user-defined functions and recursion, and perform file input-output operations.	K3
CO5	Analyse a C program by tracing and debugging it, and evaluate alternative implementations for correctness and efficiency.	K4

#Based on suggested Revised BTL

Mapping of course outcomes with program outcomes

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	H	H	M	L	H	-	-	L	L	-	M	H	-	-
CO2	H	M	H	-	H	-	-	L	L	-	M	H	-	-
CO3	H	H	H	M	H	-	-	L	L	-	M	H	-	-
CO4	H	H	H	M	H	-	-	L	L	-	M	H	-	-
CO5	H	H	M	M	H	-	-	L	L	-	H	H	-	-

(Please fill the above with Levels of Correlation, viz., L, M, H)

Lab Requirements (tools / software to carry out the experiments):

- C compiler: GCC (GNU Compiler Collection) — via MinGW-w64 on Windows, or the built-in gcc on Linux/macOS.
- IDE / editor: Code::Blocks, Dev-C++ or Visual Studio Code with the C/C++ extension.
- Command-line terminal to compile and run programs (gcc, ./a.out) and observe the compile–link–run stages.
- Debugger: GDB (GNU Debugger), integrated in Code::Blocks / VS Code, for tracing and step-by-step debugging.
- Operating system: Windows 10/11 or a Linux distribution (e.g., Ubuntu); any 64-bit PC with 4 GB RAM or more.
- No-install alternative: OnlineGDB, Programiz or Replit for quick practice and testing in a browser.

List of Experiments

Exercise 1: Getting Started with

1. Write, compile and run a C program that prints "Hello, World!" on the screen. Identify the source file, the object file and the executable produced, and record the command used at each stage.
2. Write a C program that declares variables of type int, float, char and double, reads a value for each from the user, and displays each value together with the size of its type using sizeof.
3. Write a C program to read the height and width of a rectangle from the user and compute its perimeter and are.

Exercise 2: Operators and Expressions

1. Write a C program to compute the distance between two points whose coordinates are entered by the user.
2. Write a C program that reads four integers p, q, r, s and, using relational and logical operators, prints 'Correct values' if q > r, s > p and (r + s) > (p + q); otherwise 'Wrong values'.
3. Write a C program to compute simple and compound interest for values entered by the user.

Exercise 3: Selection and Decision Making

1. Write a C program to read a character and classify it as a vowel, consonant, digit or special symbol.
2. Write a C program to find the largest of three numbers using nested if-else.
3. Write a menu-driven C program to compute the area of various geometrical shapes (circle, rectangle, triangle, square) using switch-case.

Exercise 4: Loops and Series

1. Write a C program to display the first n even natural numbers and their sum.
2. Write a C program to reverse a given integer and check whether it is a palindrome.

3. Write a C program to display the sum of the harmonic series 1 + 1/2 + 1/3 + ... up to n terms.

Exercise 5: Nested Loops and Number Patterns

1. Write a C program to print the multiplication table pyramid for numbers 1 to n using nested loops.
2. Write a C program to check whether a given number is prime, then extend it to print all prime numbers and all Armstrong numbers below.

Exercise 6: One-Dimensional Arrays

1. Write a C program to print all unique elements of an array, and implement linear search to find a given element in the array.
2. Write a C program to separate the odd and even elements of an array into two different arrays, and implement binary search to find a given element in a sorted array.
3. Write a C program to sort the elements of an array in ascending order using selection sort, and compare the efficiency of linear search vs. binary search on the sorted array by counting the number of comparisons each method takes.

Exercise 7: Matrices

1. Write a C program to perform addition of two square matrices and display the resultant matrices.
2. Write a C program to multiply two square matrices and display the resultant matrices.
3. Write a C program to find the transpose of a given matrix.

Exercise 8: Strings:

Write C programs to perform the following without using string handling functions

- **Find the length** of a given string.
- **Copy** one string into another.
- **Concatenate** two strings.
- **Compare** two strings and display whether they are equal, or which one is greater.
- **Reverse** a given string (using a separate swap function to exchange characters).
- **Search for a character** in a string and display its position (or indicate if not found).
- **Count the number of vowels, consonants, and words** in a given string

Exercise 9: Pointers Fundamentals

1. Write a C program to demonstrate the use of the & (address-of) and * (value-at-address) operators.
2. Write a C program to add two numbers using pointers.
3. Write a C program to swap two values using call by reference.

Exercise 10: Pointers and Dynamic Memory

1. Write a C program to dynamically allocate memory for an array using malloc, find the largest element and the sum of all elements in it, then repeat using calloc and note the difference between the two.
2. Write a C program to count the number of vowels in a string using a pointer.

Exercise 11: User-Defined Functions

1. Write a C program to check whether a number is prime using a user-defined function.
2. Write a C program to compute the GCD and LCM of two numbers using functions.
3. Write a C program to demonstrate a function that returns a pointer.

Exercise 12: Recursion

1. Write a C program to compute the factorial and the n-th Fibonacci term using recursion.
2. Write a C program to compute the sum of digits of a number using recursion.
3. Write a C program to convert a decimal number to its binary equivalent using a recursive function.

Exercise 13: File Handling

1. Write a C program to append multiple lines at the end of an existing text file.
2. Write a C program to copy the contents of one file into another file.
3. Write a C program to count the number of characters, words and lines in a text file.

Web References:

- NPTEL – Programming in C, IIT Kanpur: https://onlinecourses.nptel.ac.in/noc21_cs81/preview
- Virtual Labs – Problem Solving Lab, IIIT Hyderabad: https://ps-iiith.vlabs.ac.in/
- GeeksforGeeks – C Programming: https://www.geeksforgeeks.org/c-programming-language/

Course Code	DIGITAL LOGIC DESIGN	L	T	P	Credits
		2	0	0	2

Pre-requisite:

Course Objectives

- Develop proficiency in number systems, codes, and Boolean algebra as the mathematical basis of digital logic.
- Enable students to analyze and design combinational logic circuits using minimization techniques and standard building blocks.
- Introduce sequential logic elements such as latches and flip-flops and their role in storing digital state.
- Build the ability to design synchronous sequential circuits, including registers, counters, and finite state machines.

CO1	Explain number systems, binary arithmetic, complements, binary codes, Boolean algebra, and the operation of basic and universal logic gates.	K2
CO2	Apply Boolean algebra and Karnaugh-map techniques to simplify Boolean expressions and design basic combinational circuits such as adders and subtractors.	K3
CO3	Analyze and design combinational logic circuits using multiplexers, demultiplexers, encoders, decoders, and magnitude comparators.	K4
CO4	Explain and analyze latches and flip-flops, including SR, D, JK, and T flip-flops, using excitation tables and timing diagrams.	K4
CO5	Design and evaluate sequential circuits using shift registers and asynchronous, synchronous, up/down, and modulo-N counters	K5

Mapping of course outcomes with program outcomes

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	2	1	-	-	-	-	-	-	-	-			
CO2	3	3	3	2	2	-	-	-	-	-	-			
CO3	3	3	3	2	2	-	-	-	-	-	-			
CO4	3	3	2	2	2	-	-	-	-	-	-			
CO5	3	3	3	3	3	-	-	-	-	-	2			

UNIT	COURSE CONTENT	Contact Hours
UNIT-I	Number Systems and Boolean Algebra Number systems (binary, octal, decimal, hexadecimal) and inter-conversions; binary arithmetic; complements (1's and 2's); binary codes (BCD, Gray code, ASCII); Boolean algebra postulates and theorems; logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR) and universal gates	**7 Hrs.**
UNIT-II	Combinational Logic Design I (8 Hours) Minimization using Karnaugh maps (up to 4 variables); Sum-of-Products and Product-of-Sums forms; design of half-adder, full-adder, half-subtractor, full-subtractor	**5 Hrs.**
UNIT-III	Combinational Logic Design II: multiplexers and demultiplexers; encoders and decoders; magnitude comparators. Sequential Logic Fundamentals- Latches and FlipFlops, Difference between combinational and sequential circuits;	**5 Hrs.**
UNIT-IV	Sequential Logic Design—-SR latch; clocked SR, D, JK, and T flip-flops; master-slave flip-flop; flip-flop excitation tables; timing diagrams; setup and hold time.	**5 Hrs.**
UNIT-V	Registers, Counters, and Sequential Circuit Design (7 Hours) Shift registers (SISO, SIPO, PISO, PIPO); ripple (asynchronous) counters; synchronous counters; up/down and modulo-N counters	**5 Hrs.**

Text Books

1. Mano, M. M., Ciletti, M. D. (2018). Digital design (6th ed.). Pearson.
2. Floyd, T. L. (2015). Digital fundamentals (11th ed.). Pearson.

Reference Books

1. Hill, F. J., & Peterson, G. R. (2010). Introduction to switching theory and logic design (3rd ed.). Wiley.
2. Wakerly, J. F. (2018). Digital design: Principles and practices (5th ed.). Pearson.
3. Brown, S., &Vranesic, Z. (2014). Fundamentals of digital logic with VHDL design (3rd ed.). McGraw-Hill Education.

Web References

1. NPTEL. (n.d.). Digital circuits and systems. National Programme on Technology Enhanced Learning. https://nptel.ac.in/
2. MIT OpenCourseWare. (n.d.). Digital systems laboratory. Massachusetts Institute of Technology. https://ocw.mit.edu/courses/6-111-introductory-digital-systems-laboratory-fall-2019/
3. All About Circuits. (n.d.). Digital circuits. https://www.allaboutcircuits.com/textbook/digital/

COURSE CODE –	IT and Electronics Workshop	CATEGORY ESC	L-T-P-C 0-0-3-1.5

Pre-requisites: NIL

Course Objectives (CO):

- Familiarize students with computer hardware, operating systems, peripheral devices and software installation.
- Develop practical skills in computer networking, internet technologies and collaborative platforms.
- Introduce students to the Linux command-line environment, shell scripting and version control systems.
- Develop hands-on skills in basic electronics, circuit assembly, soldering, measurement, testing and troubleshooting.
- Develop an industry-oriented mini hardware prototype by integrating basic electronics, sensors, embedded software and appropriate documentation.

Course Outcomes (CO): *After successful completion of the course, the student will be able to*:

CO1: Assemble, configure and troubleshoot computer hardware, operating systems and software systems (K3).

CO2: Configure wired and wireless networks and use internet, cloud collaboration and related digital tools (K3).

CO3: Use Linux commands, shell utilities and Git/GitHub for basic software development and collaboration workflows (K3).

CO4: Identify electronic components and instruments, assemble basic electronic circuits, perform soldering, make electrical measurements and troubleshoot simple circuits (K3).

CO5: Develop and demonstrate a working mini hardware prototype integrating electronic components, sensors and embedded software while following basic testing, documentation and presentation practices (K5).

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	H	M	-	-	H	-	-	-	M	L	L
CO2	M	H	-	M	H	-	-	-	M	M	L
CO3	M	M	M	-	H	-	-	-	M	M	M
CO4	H	H	H	M	H	-	-	-	M	M	M
CO5	H	H	H	H	H	L	L	M	H	H	H

List of Experiments:

1. PC Hardware Component Identification and Testing
2. Desktop Computer Assembly
3. Windows/Linux Installation and Basic Troubleshooting
4. LAN Configuration and Network Cable Testing
5. Wi-Fi Router Configuration
6. Basic Linux Commands and File Management
7. Bash Scripting Basics
8. Git Repository Creation and Basic Version Control
9. GitHub Collaboration
10. Identification and Testing of Basic Electronic Components
11. Basic Electronic Circuit Assembly Using Breadboard
12. Soldering and Desoldering Practice
13. Measurement and Testing Using a Digital Multimeter
14. Construction and Troubleshooting of a Simple Electronic Circuit
15. Mini Project Development and Demonstration

Learning Resources:

Text Books:

1. V. K. Mehta and Rohit Mehta, Principles of Electronics, S. Chand & Company.
2. B. S. Raghuwanshi, A Course in Workshop Technology, Dhanpat Rai & Sons.
3. Vishnu P. Singh, Computer Hardware Course, Asian Publishers/Computech Publications.

Online Resources:

- **NPTEL – Computer Networks** – for networking fundamentals and practical concepts.
- **NPTEL – Computer Networks and Internet Protocol, IIT Kharagpur** – for TCP/IP and Internet fundamentals.
- **Ubuntu Official Documentation** – for Linux installation, file management, networking and system administration.
- **NPTEL – Basic Electronics, IIT Bombay** – for basic electronic circuits and components.
- **NPTEL – Basic Electronics and Lab, IIT Madras** – particularly useful for the practical electronics component.
- **GitHub Learning Resources** – for Git/GitHub workflows.
- **GitHub Skills** – for hands-on Git/GitHub exercises.
- **Official Git Documentation** – as a reference for version control.
- **Tinkercad Circuits** – for virtual breadboard and circuit prototyping.
- **Arduino Documentation** – for the mini-project/embedded-prototyping component.

Course Code	DESIGN THINKING & INNOVATION	L	T	P	Credits
		0	1	3	2.5

Course Objectives

- Introduce students to the principles, mindset and methodologies of **Design Thinking and human-centered problem solving**.
- Develop students' ability to **observe, empathize, analyze user needs and formulate well-defined engineering and societal problems**.
- Develop students' **creative and innovative thinking abilities** through systematic ideation, concept generation and evaluation techniques.
- Enable students to **develop, prototype, test and iteratively improve solutions** using user feedback and appropriate physical/digital tools.
- Develop students' ability to **identify innovation opportunities, create value propositions, understand basic IPR and communicate innovative solutions effectively**.

CO1	Explain the principles, processes and mindset of Design Thinking for engineering problem-solving and innovation	K2
CO2	Apply empathy, observation and problem-framing techniques to identify user needs and define engineering problems	K3
CO3	Apply creative and systematic ideation techniques to generate and select innovative solution concepts	K3
CO4	Analyze solution concepts and prototypes using user feedback and appropriate design criteria for iterative improvement	K4
CO5	Analyze the innovation potential of solutions considering value proposition, feasibility, sustainability, societal impact and basic IPR aspects	K4

Mapping of course outcomes with program outcomes

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	2	1	–	–	1	–	–	1	1	2			
CO2	2	3	3	1	1	3	2	1	2	2	2			
CO3	1	2	3	1	2	1	1	–	3	2	2			
CO4	3	3	3	2	3	2	2	1	3	2	2			
CO5	1	2	3	1	2	3	3	2	2	3	3			

UNIT	COURSE CONTENT	Contact Hours
UNIT-I	**Design Thinking Foundations & Empathy** Introduction to Design Thinking ; Design thinking mindset and principles ; Design thinking vs. conventional engineering problem-solving ; Human-centered design ; Five-stage design thinking process ; Creativity, curiosity and observation ; Understanding users and stakeholders ; Empathy and user research ; Interviews, observation and contextual inquiry ; Empathy mapping ; Personas and user journeys ; Identifying explicit and latent/unarticulated needs ; Ethical considerations in user research **Practical Activities** • Observation of a real-life problem • Conducting user interviews • Preparing an empathy map • Developing a persona • Preparing a user journey map **Unit Outcome** Students will be able to **understand users, identify their needs and translate observations into meaningful design opportunities**.	
UNIT-II	**Problem Definition, Analysis & Opportunity Identification** From observations to insights ; Data synthesis and affinity mapping ; Identifying patterns and pain points ; Problem statements ; Point-of-View (POV) statements ; "How Might We" (HMW) questions ; Stakeholder mapping ; 5 Whys and root-cause analysis ; Problem prioritization ; Need–Opportunity mapping ; Problem validation ; Design constraints and specifications ; Social, environmental and sustainability considerations ; Identifying opportunities for innovation **Practical Activities** • Affinity mapping of interview data • Develop three alternative problem statements • Prepare POV and HMW statements • Stakeholder map • Root-cause analysis • Select and validate one problem/opportunity **Unit Outcome**	

	Students will be able to **analyze user research, identify root causes, frame well-defined problems and recognize opportunities for innovative solutions**.	
UNIT-III	**Creativity, Ideation & Concept Development** Creativity and innovation mindset; Divergent and convergent thinking ; Brainstorming and brainwriting ; SCAMPER ; Mind mapping ; Reverse brainstorming ; Analogical thinking ; Biomimicry ; Lateral thinking ; Forced connections ; Morphological analysis ; TRIZ — basic principles ; Individual and collaborative ideation ; AI-assisted ideation: opportunities and limitations ; Idea clustering and screening ; Impact–feasibility matrix ; Concept selection ; Design criteria and decision matrix **Practical Activities** • Generate 20–30 ideas for a selected problem • SCAMPER exercise • Brainwriting exercise • Biomimicry-based ideation • AI-assisted idea generation and critical evaluation • Prepare an idea-selection matrix • Select the top 2–3 concepts **Unit Outcome** Students will be able to **generate diverse solution concepts using creative techniques and systematically select promising ideas based on defined criteria**.	
UNIT-IV	**Prototyping, Experimentation & Validation** Concept development ; Prototype mindset ; Types of prototypes: Paper prototype, Physical prototype, Digital prototype, Storyboard, Role-play, Service prototype ; Low-fidelity and high-fidelity prototypes ; Rapid prototyping ; "Fail fast, learn fast" ; Minimum Viable Prototype (MVP) ; Design experimentation ; Prototype evaluation ; User testing ; Think-aloud method ; Feedback collection ; Usability and functionality testing; Iterative design ; Design critique ; Documentation of iterations ; Use of CAD, simulation and AI-assisted visualization tools **Practical Activities** • Develop a low-fidelity prototype • Prepare a storyboard • Build a physical/digital prototype • Conduct user testing • Collect structured feedback • Modify the prototype • Prepare an iteration log	
UNIT-V	**Innovation, Value Creation & Communication** **Innovation: meaning, characteristics and importance:** Invention vs. innovation ; Incremental, disruptive and radical innovation ; Product, process, service and social innovation ; Technology-driven vs. need-	

	driven innovation ; Sustainable and frugal innovation ; Innovation opportunities and emerging technologies ; From prototype to innovation ; Desirability, feasibility and viability ; Value Proposition ; Value Proposition Canvas ; Introduction to Business Model Canvas ; Customer segments and stakeholder value; Innovation validation; Market/user validation ; Intellectual Property Rights — introduction; Patents, copyright, trademarks and design protection ; Responsible innovation ; Inclusive and sustainable innovation ; Innovation ecosystem and entrepreneurship ; Communicating innovative ideas ; Innovation pitch **Practical Activities** 1. Classify existing products/services as incremental, disruptive or radical innovations. 2. Identify three innovation opportunities from a campus/community problem. 3. Prepare a **Value Proposition Canvas**. 4. Validate the proposed innovation with potential users. 5. Prepare an introductory **Business Model Canvas**. 6. Identify possible IPR protection. 7. Prepare and deliver a **3-minute Innovation Pitch**.	

Text Books

1. Chandan Adhikari, Prabir Kumar Das & Tanay Pramanik, Textbook of Design Thinking and Innovation, S. Chand Publishing, 2026.
2. Shalini Rahul Tiwari & Rohit Rajendra Swarup, Design Thinking: A Comprehensive Textbook, Wiley India, 2024

Reference Books

1. Soni, Pavan, Design Your Thinking: The Mindsets, Toolsets and Skill Sets for Creative Problem-Solving, Penguin Random House India.
2. Jauhari, Vinnie & Sudhanshu Bhushan, Innovation Management, Oxford University Press India.

Web References

1. Atal Innovation Mission, NITI Aayog – Innovation and Design Thinking Resources. https://aim.gov.in/
2. Institution's Innovation Council (IIC), Ministry of Education, Government of India, https://iic.mic.gov.in/
3. Startup India, Government of India – Innovation and Entrepreneurship Ecosystem, https://www.startupindia.gov.in/
4. National Innovation Foundation – India – Grassroots and Frugal Innovation, https://nif.org.in/
5. Stanford d.school – Design Thinking Bootleg and Design Thinking Resources, https://dschool.stanford.edu/resources/design-thinking-bootleg

6. IDEO.org – Human-Centered Design and Social Innovation Resources, https://www.ideo.org/
7. IDEO U – Design Thinking and Innovation Resources, https://www.ideou.com/pages/design-thinking
8. NPTEL – Online Courses on Design, Innovation, Creativity and Entrepreneurship, https://nptel.ac.in/
9. SWAYAM – Online Courses on Innovation, Entrepreneurship and Design Thinking, https://swayam.gov.in/

COURSE CODE – R26XXXX	DATA STRUCTURES USING C	ESC	L-T-P 3-0-0	CREDITS: 3

Pre-requisite: Computational Thinking and Problem Solving using C

Course Objectives: At the end of the course, the teacher should be able to:

- Understand the fundamental concepts of algorithms, structured programming, and abstract data types using the C programming language.
- Apply programming constructs and basic data structures to organize and manipulate data efficiently.
- Develop problem-solving skills by implementing linear and non-linear data structures using C.
- Analyze the performance of searching, sorting, and traversal algorithms using basic complexity measures.
- Apply appropriate data structures to solve computational problems encountered in engineering applications.

Course Outcomes: *At the end of the course, student will be able to*

CO	Course Outcome Description	Knowledge Level (K)#
CO1	Explain fundamental data structure concepts, abstract data types, classifications, and analyze the time and space complexity of algorithms.	K2
CO2	Implement and evaluate searching and sorting algorithms, including linear search, binary search, bubble sort, selection sort, insertion sort, merge sort, and quick sort, based on their computational efficiency.	K3
CO3	Design and implement linear data structures such as linked lists, stacks, and queues using arrays, pointers, and dynamic memory allocation, and apply them to solve computational problems.	K4
CO4	Implement and apply tree data structures, including binary trees, binary search trees, and heaps, using appropriate traversal and manipulation techniques.	K3
CO5	Represent and process graphs and hash tables using appropriate techniques, and implement graph traversal and minimum spanning tree algorithms along with hashing and collision-resolution methods.	K3

#Based on suggested Revised BTL

Mapping of course outcomes with program outcomes

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO 2	PS O3
CO1	H	M	-	M	M	-	-	-	L	-	M	H	L	L
CO2	H	H	M	M	H	-	-	-	L	-	M	H	L	L
CO3	H	H	H	M	H	-	-	M	L	L	M	H	L	L
CO4	H	H	H	M	H	-	-	M	L	L	M	H	L	L
CO5	H	H	H	M	H	L	-	M	L	L	H	H	L	L

(Please fill the above with Levels of Correlation, viz., L, M, H)

UNIT	CONTENTS	Contact Hours
UNIT – 1	Introduction to data Structures, Abstract Data Types (ADT); Need and classification of Data Structures; Introduction to algorithm analysis; Time and space complexity (basic concepts). **Searching and Sorting:** Linear Search; Binary Search; Bubble Sort; Selection Sort; Insertion Sort; Merge Sort; Quick Sort;	9
UNIT – 2	**Linear Data Structures:** Dynamic memory allocation; Recursion; Linked Lists—Singly, Doubly, and Circular Linked Lists; Basic operations (creation, insertion, deletion, traversal, etc.); Applications of Linked Lists;	9
UNIT – 3	**Stacks**—Array and Linked List implementations; operations on stacks, Basic applications – infix to postfix conversion, postfix expression evaluation, parenthesis matching; **Queues**— Array and Linked List implementations; Simple Queue, Circular Queue, operations and Applications of Queues.	9
UNIT – 4	**Trees:** Introduction and terminology; Binary Trees; Tree representation; Types of Binary Trees; Tree traversals; Binary Search Trees—Insertion, Deletion, Searching; Heap—Min Heap and Max Heap; Applications of Trees.	9
UNIT – 5	**Graphs:** Graph terminology; Graph representations (Adjacency Matrix and Adjacency List); Graph traversals—Breadth First Search (BFS) and Depth First Search (DFS); Introduction to Minimum Spanning Trees (Prim's Algorithm); Applications of Graphs. **Hashing:** Introduction to Hashing; Hash Functions; Collision Resolution, open addressing: separate chaining, closed addressing: linear probing, quadratic probing, double hashing, rehashing, examples.	9
	Total	**45**

Text Books:

1. Balagurusamy, E. (2025). Data Structures Using C (2nd ed.). McGraw Hill Education India.
2. Thareja, R. (2023). Data Structures Using C (3rd ed.). Oxford University Press India.

Reference Books:

1. Deshpande, P. S., & Kakde, O. G. C and data structures. Dreamtech Press.
2. Shukla, R. K. Data structures using C and C++. Wiley India.

Web Resources:

- National Programme on Technology Enhanced Learning. (2026). Introduction to Programming in C. IIT Kanpur. https://nptel.ac.in/courses/106104128
- National Programme on Technology Enhanced Learning. (2026). Programming, Data Structures and Algorithms using C. IIT Madras. https://nptel.ac.in/courses/106106127
- National Programme on Technology Enhanced Learning. (2026). Problem Solving Through Programming in C. IIT Kharagpur. https://onlinecourses.nptel.ac.in
- Virtual Labs: https://ds1-iiith.vlabs.ac.in/
- Programiz. Data Structures. https://www.programiz.com/dsa
- Visualgo. Data Structure and Algorithm Visualizations. https://visualgo.net/en

COURSE CODE – R26XXXX	DATA STRUCTURES USING C LAB	ESC	L-T-P 0-0-3	CREDITS: 1.5

Pre-requisite: Programming for Problem Solving using C

Course Objectives: The main objectives of the course are to

- Build hands-on programming proficiency in C and to train students to translate the design of a data structure into a correct, efficient working program.
- Strengthen the ability to trace, debug and test code, and to choose an appropriate data structure and implementation strategy for a given problem

Course Outcomes: *At the end of the course, student will be able to*

CO	Course Objective Description	Knowledge Level (K)#
CO1	Write and debug C programs using pointers and dynamic memory allocation to implement array-based searching and sorting techniques.	K3
CO2	Build and manipulate singly, doubly and circular linked lists, and use them to solve problems such as polynomial arithmetic and duplicate removal.	K3
CO3	Implement stack-based and queue-based programs for expression conversion/evaluation, parenthesis matching and recursion-related problems.	K3
CO4	Construct binary search trees and implement all traversal techniques to solve hierarchical-data problems.	K3
CO5	Design and test hash-table programs using chaining and open-addressing collision-resolution techniques.	K5

#Based on suggested Revised BTL

Mapping of course outcomes with program outcomes

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	H	H	M	-	H	-	-	L	L	-	-	H	L	L
CO2	H	M	H	-	H	-	-	L	L	-	-	H	L	L
CO3	H	H	M	-	H	-	-	-	L	-	-	H	L	L
CO4	H	H	H	M	H	-	-	L	L	-	L	H	L	L
CO5	H	H	M	-	H	-	-	L	L	-	-	H	L	L

Lab Requirements (tools / software to carry out the experiments):

- C compiler: GCC (MinGW-w64 on Windows / built-in on Linux) with an IDE such as Code::Blocks,
- VS Code or Dev-C++.
- Debugger: GDB, to trace pointers, linked lists and recursion step-by-step.

- Memory checker: Valgrind (on Linux) to detect memory leaks and invalid access in dynamic-memory programs.
- Visualization aids: VisuAlgo, USF Data-Structure Visualizations, or the IIIT-H Virtual Labs for lists,
- trees and hashing.
- Operating system: Windows or Linux, 64-bit, with 4 GB RAM or more.
- No-install alternative: OnlineGDB or Replit for quick compilation and sharing.

List of Experiments

Exercise 1: Array-Based Techniques, searching

1. Write a program using pointers to reverse an array in place, without using a second array.
2. Write a program to dynamically allocate an array using malloc, resize it using realloc, and free it correctly.
3. Implement linear search on an unsorted array; count and print the number of comparisons made.
4. Implement binary search on a sorted array; count the comparisons made and compare the count with linear search on the same data.

Exercise 2: Sorting

1. Implement bubble, selection and insertion sort; print the array after each pass, and tabulate the comparisons and swaps each algorithm performs on the same input.
2. Implement Quick sort and Merge sort; print the array after every partition/merge step to observe how each algorithm progresses.

Exercise 3: Singly Linked List

1. Implement a singly linked list with creation, traversal, insertion and deletion of nodes.
2. Implement a singly linked list with insertion, deletion operations at the beginning, end and a given position.
3. Implement deletion of a node by value and by position with proper handling of an empty list, then reverse the list iteratively and recursively.

Exercise 4: Singly Linked List Applications

1. Detect and remove duplicate nodes from an unsorted linked list.
2. Find the middle node of a singly linked list in a single traversal.
3. Merge two sorted singly linked lists into a single sorted list.

Exercise 5: Doubly and Circular Linked Lists

1. Implement a doubly linked list with insertion, deletion and forward/backward traversal.
2. Implement a circular linked list and demonstrate insertion, deletion and traversal without infinite looping.
3. Represent two polynomials using linked lists and write a program to add them.

Exercise 6: Stack Implementation

1. Implement a stack using an array and using a linked list, with overflow/underflow checks.
2. Write a program to check whether an expression has balanced parentheses using a stack.

Exercise 7: Stack Applications

1. Write a program to convert an infix expression to postfix using a stack.
2. Write a program to evaluate a postfix expression using a stack.
3. Write a program to check whether a given string is a palindrome using a stack.

Exercise 8: Recursion and Stack Simulation

1. Write recursive programs for factorial, Fibonacci and sum of digits.
2. Simulate the run-time call stack of a recursive function by writing an equivalent iterative version using an explicit stack.

Exercise 9: Queue and Circular Queue

1. Implement a circular queue and demonstrate that it avoids the false-overflow problem of a linear queue.
2. Implement a queue using an array and using a linked list.

Exercise 10: Binary Trees

1. Implement a Binary Tree with insertion and search operations.
2. Implement deletion of a node from a binary tree, handling all three cases (leaf, one child, two children).

Exercise 11: Binary Search Trees

1. Implement a Binary Search Tree (BST) with insertion and search operations.
2. Implement deletion of a node from a BST, handling all three cases (leaf, one child, two children).

Exercise 11: Tree Traversals and Problems

1. Implement a Binary Search Tree (BST) with insertion, and perform in-order, pre-order and post-order traversals.
2. Write a program to find the height of a binary tree and count the total number of nodes and leaf nodes.
3. Write a program to check whether a given binary tree is a valid Binary Search Tree.

Exercise 12: Hashing

1. Implement a hash table using chaining for collision resolution, with insertion, search and deletion.
2. Implement a hash table using linear probing.

Web References:

1. **NPTEL – Programming, Data Structures and Algorithms using C (IIT Madras)**
   https://www.nptel.ac.in/courses/106106127
2. **NPTEL – Data Structures and Algorithms (IIT Delhi)**
   https://www.nptel.ac.in/courses/106102064
3. **NPTEL – Programming, Data Structures and Algorithms using C – Course Archive**
   https://onlinecourses-archive.nptel.ac.in/noc18_cs25/preview
4. **Virtual Labs – Data Structures I (IIIT Hyderabad)**
   https://ds1-iiith.vlabs.ac.in/
5. **Virtual Labs – IIIT Hyderabad Data Structures Labs**
   https://www.vlab.co.in/participating-institute-iiit-hyderabad
6. **GeeksforGeeks – Data Structures and Algorithms**
   https://www.geeksforgeeks.org/dsa/

COURSE CODE – R26XXXX	PYTHON PROGRAMMING	ESC	L-T-P 2-0-0	CREDITS: 2

Pre-requisite: Problem Solving and Programming Using C.

Course Objectives: The main objectives of the course are to

- Develop proficiency in Python control structures while explicitly linking conditionals to abstraction and loops to pattern recognition.
- Build competence in selecting Python data structures (strings, lists, tuples, dictionaries)
- Introduce object-oriented programming concepts such as classes, objects, inheritance, and polymorphism, enabling students to model real-world problems through encapsulation and abstraction.
- Train students to decompose problems into functions and recursive sub-problems, use lambda expressions and functional tools, reason informally about algorithmic efficiency, and use AI coding assistants responsibly.
- Equip students with file handling, exception handling, and basic data analysis/visualisation skills, framed as decomposing a larger data workflow into manageable, robust steps and to integrate these skills in a capstone mini-project.

Course Outcomes: *At the end of the course, student will be able to*

CO	Course Outcome Description	Knowledge Level (K)#
CO1	Develop Python programs using variables, data types, operators, input/output, conditional statements, loops, and appropriate coding practices to solve computational problems.	K3
CO2	Implement Python programs using strings, lists, sets, tuples, dictionaries, and comprehensions to efficiently represent, manipulate, and process different types of data.	K3
CO3	Develop and analyze modular Python programs using functions, lambda expressions, functional programming tools, recursion, modules, and libraries, considering basic algorithmic efficiency.	K4
CO4	Design and implement object-oriented Python programs using classes, objects, encapsulation, inheritance, polymorphism, operator overloading, and composition to model real-world problems.	K4
CO5	Develop Python applications that perform file processing and exception handling, and apply NumPy, pandas, and Matplotlib for basic data analysis and visualization.	K3

#Based on suggested Revised BTL

Mapping of course outcomes with program outcomes

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	H	M	M	-	H	-	-	L	L	-	M	H	L	L
CO2	H	H	H	M	H	-	-	L	L	-	M	H	L	L
CO3	H	H	H	M	H	-	-	M	L	-	H	H	L	L
CO4	H	H	H	M	H	-	-	M	M	L	H	H	L	L
CO5	H	H	H	H	H	M	-	M	M	L	H	H	L	L

UNIT	CONTENTS	Contact Hours
UNIT – 1	**PYTHON BASICS AND CONTROL FLOW** **Python Basics:** Translating algorithms and flowcharts into Python, variables, data types, operators and input/output. Good Coding Practice: Docstrings, inline comments and print-based debugging (threaded through every unit from here on). **Conditional Statements:** if, if-else and chained/nested conditionals, abstraction over multiple cases. **Loops:** for and while loops, pattern recognition of repetition. **Loop Control:** Nested loops and loop control (break, continue, pass), decomposition of complex repetitive logic.	
UNIT – 2	**DATA STRUCTURES IN PYTHON** **Strings**: Pattern matching and abstraction of text as a sequence; string operations and methods. **Lists:** Abstraction of ordered, mutable collections; list operations; list comprehensions. **Sets:** Set operations and Applications. **Tuples and Dictionaries:** Abstraction of fixed (tuples) and key-based (dictionaries) data; common operations and use-case selection.	
UNIT – 3	**FUNCTIONS, RECURSION AND MODULES** **Functions:** Decomposition of a program into independent, reusable sub-problems; parameters, return values and scope. **Lambda Expressions and Functional Tools:** lambda, map, filter, and reduce, concise transformations and their use cases. **Recursion:** Algorithmic thinking applied to self-similar sub-problems. Algorithmic Efficiency: Informal comparison of iterative vs. recursive approaches and their time trade-offs. **Modules and Libraries:** Importing and creating modules, packages and built-in libraries; responsible AI-assisted coding practices.	
UNIT – 4	**OBJECT ORIENTED PROGRAMMING** **Introduction to OOP**: Features of object-oriented programming, encapsulation, abstraction, inheritance, polymorphism.	

	Classes and Objects: Class definition, the __init__ method, class variables and instance variables, public and private data members, calling methods from other methods, built-in class attributes. **Inheritance:** Inheriting classes, types of inheritance, method overriding. **Polymorphism and Operator Overloading:** Polymorphism through method overriding; operator overloading. **Composition:** Composition (has-a) vs. inheritance (is-a) relationships.	
UNIT – 5	**FILE HANDLING, EXCEPTIONS AND DATA ANALYSIS** **File Handling:** Reading and writing files; decomposition of a data workflow into read/process/write stages. **Exception Handling**: try-except-finally; abstraction of error cases into predictable, manageable patterns; user-defined exceptions. **Data Analysis and Visualization**: Introduction to NumPy, pandas and Matplotlib basics; pattern recognition in real datasets.	
	Total	

Text Books:

1. Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.
2. J. V. Guttag, Introduction to Computation and Programming Using Python: With Application to Computational Modeling and Understanding Data, 3rd Edition, MIT Press, 2021.

Reference Books:

1. Sweigart, A. (2025). Automate the boring stuff with Python: Practical programming for total beginners (3rd ed.). No Starch Press.
2. McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.
3. J. M. Wing, Computational Thinking, Communications of the ACM, 49(3), 33–35, 2006. https://doi.org/10.1145/1118178.1118215
4. Jeeva Jose, "Taming Python by Programming", 2nd Edition, Khanna Book Publishing Co., Delhi, 2019.

Web References:

1. Python Software Foundation. (n.d.). The Python tutorial. https://docs.python.org/3/tutorial/
2. W3Schools. (n.d.). Python tutorial. https://www.w3schools.com/python/
3. NPTEL. (n.d.). Programming, data structures and algorithms using Python. https://nptel.ac.in
4. Real Python. (n.d.). Python tutorials. https://realpython.com/

COURSE CODE – R26XXXX	PYTHON PROGRAMMING LAB	ESC	L-T-P 0-0-2	CREDITS: 1.0

Pre-requisite: Concurrent with Python Programming

Course Objectives: The main objectives of the course are to

- Develop proficiency in Python control structures while explicitly linking conditionals to abstraction and loops to pattern recognition.
- Build competence in selecting Python data structures (strings, lists, tuples, dictionaries)
- Introduce object-oriented programming concepts such as classes, objects, inheritance, and polymorphism, enabling students to model real-world problems through encapsulation and abstraction.
- Train students to decompose problems into functions and recursive sub-problems, use lambda expressions and functional tools, reason informally about algorithmic efficiency, and use AI coding assistants responsibly.
- Equip students with file handling, exception handling, and basic data analysis/visualisation skills, framed as decomposing a larger data workflow into manageable, robust steps and to integrate these skills in a capstone mini-project.

Course Outcomes : *At the end of the course, student will be able to*

CO	Course Outcome Description	Knowledge Level (K)#
CO1	Develop Python programs using variables, data types, operators, input/output, conditional statements, loops, and appropriate coding practices to solve computational problems.	K3
CO2	Implement Python programs using strings, lists, tuples, dictionaries, and comprehensions to efficiently represent, manipulate, and process different types of data.	K3
CO3	Develop and analyze modular Python programs using functions, lambda expressions, functional programming tools, recursion, modules, and libraries, considering basic algorithmic efficiency.	K4
CO4	Design and implement object-oriented Python programs using classes, objects, encapsulation, inheritance, polymorphism, operator overloading, and composition to model real-world problems.	K4
CO5	Develop Python applications that perform file processing and exception handling, and apply NumPy, pandas, and Matplotlib for basic data analysis and visualization.	K3

#Based on suggested Revised BTL

Mapping of course outcomes with program outcomes

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	H	M	M	-	H	-	-	L	L	-	M	H	L	L
CO2	H	H	H	M	H	-	-	L	L	-	M	H	L	L
CO3	H	H	H	M	H	-	-	M	L	-	H	H	L	L
CO4	H	H	H	M	H	-	-	M	M	L	H	H	L	L
CO5	H	H	H	H	H	M	-	M	M	L	H	H	L	L

Lab Requirements (tools / software to carry out the experiments):

- Python 3.x interpreter (CPython, from python.org).
- IDE / editor: Thonny or IDLE (beginner-friendly), VS Code (Python extension) or PyCharm Community.
- pip package manager with the NumPy, pandas and Matplotlib libraries for the data-analysis exercises.
- Notebook environment: Jupyter Notebook or Google Colab (no install) for experimentation and plotting.
- Operating system: Windows, Linux or macOS, 64-bit, with 4 GB RAM or more.
- No-install alternative: Google Colab, Replit or Programiz Python for practice in a browser.

List of Experiments

Exercise 1: Getting Started with Python

1. Write and run a Python program that prints "Hello, World!" along with the Python version in use. Note the difference between running a saved script from the terminal and typing the same statements in the interactive shell.
2. Write a Python program that reads a name and an age from the user and prints a personalised greeting.
3. Write a Python program that reads two numbers and prints their sum, difference, product, quotient and remainder, each on a labelled line.
4. Write a Python program to compute simple interest given principal, rate, and time.
5. Write a Python program to compute the area and perimeter of a rectangle from user input.
6. Write a Python program to swap two numbers without using a third variable.

Exercise 2: Conditionals

1. Write a Python program to compute the Body Mass Index (BMI) from height and weight, rounded to two decimal places, and classify it as Underweight, Normal, Overweight or Obese using if-elif-else.
2. Write a menu-driven Python program to convert a temperature between Celsius, Fahrenheit and Kelvin based on the user's choice.
3. Write a Python program to find the largest of three numbers.
4. Write a Python program to Check whether a given year is a leap year.

Exercise 3: Write a Python program to

1. Determine whether a number is positive, negative, or zero.
2. Check if a given number is prime.
3. Convert temperature between Celsius and Fahrenheit based on user choice.
4. Compute and display the area of a valid triangle using Heron's formula, with inline comments explaining each step.

Exercise 4: Loops, Number Problems and Patterns

1. Use a for loop to print the first n Fibonacci terms, and a while loop to test whether a number is prime (using break, printing the number of division tests).
2. Use a while loop to compute the GCD and LCM of two integers using Euclid's algorithm.
3. Find the first n prime numbers using nested loops with break.
4. Sum of first N natural numbers.

Exercise 5: String Processing

1. Write a function that checks whether a string is a palindrome using slicing and string methods, ignoring spaces, punctuation and case; justify whether a string or a list of characters is the better structure.
2. Write a script that counts the vowels, consonants, digits and special characters in a user-provided paragraph.
3. Write a script that sanitizes a full name with irregular capitalization and excess spaces using .strip() and .title(), and extracts the initials.

Exercise 6: Lists and List Comprehensions

1. Remove duplicates from a list of sensor readings, first without set() and then with set(); compare the two approaches for time complexity and readability.
2. Use a list comprehension to filter out noise readings outside a realistic range (e.g., -40°C to 85°C).
3. Compute and display the average, maximum and minimum of the filtered list.

Exercise 7: Write a Python Program to

1. Find the second-largest element in a list of exam scores.
2. Merge two sorted lists of student roll numbers into a single sorted list.
3. Generate squares of all even numbers from 1 to 50 using list comprehension.

Exercise 8: Dictionaries and Tuples

1. Create a dictionary of student records keyed by roll number with (Name, Marks) tuples as values; justify why a dictionary is preferable to a list of tuples for lookup.

2. Build a word-frequency dictionary from a text paragraph (case-insensitive). (Count the frequency of each word in a paragraph using a dictionary)
3. Search the student dictionary by roll number, assign grades and identify the highest scorer.

Exercise 11: Recursion vs Iteration

1. Write a recursive function fibonacci_recursive(n) that returns the n-th Fibonacci term.
2. Write an iterative function fibonacci_iterative(n) using a loop.
3. Compute the GCD of two numbers recursively using the Euclidean algorithm.
4. Compute factorial using recursion

Exercise 12: File Handling and Modular Workflow

1. Create a file student_records.txt and write five lines of data formatted as RollNo, Name, Marks.
2. Read the file, identify students scoring above 75 and write them to distinction_list.csv using context managers.
3. Count the words, lines, and characters in a given text file.
4. Read a text file of daily temperature readings and write the maximum, minimum, and average to an output file

Exercise 13: Exception Handling

1. Write a program to compute the real roots of a quadratic equation, handling all cases (real distinct, real equal, no real roots).
2. Wrap the parsing and calculation in a try-except block handling ValueError and ZeroDivisionError, with a finally block logging attempts to calculation_log.txt.
3. Package the logic into a reusable function solve_quadratic(a, b, c) suitable for import by other modules.

Exercise 14: Data Analysis and Visualization

1. Data Cleaning and Bar Chart: Fix missing values in a small dataset and show the results in a bar graph.
2. Grouping Data and Line Graph: Group data by a category and track changes over time on the given sample dataset.
3. Correlation and Scatter Plot: Check if two variables relate to each other visually and numerically on the given sample dataset.

Reference Books:

1. Sweigart, A. (2025). Automate the boring stuff with Python: Practical programming for total beginners (3rd ed.). No Starch Press.

2. McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.
3. Wing, J. M. (2006). Computational thinking. Communications of the ACM, 49(3), 33–35. https://doi.org/10.1145/1118178.1118215

Web References:

- Python Software Foundation – The Python Tutorial: https://docs.python.org/3/tutorial/
- NPTEL – Python for Data Science: https://nptel.ac.in

AI Tools and Applications

For All Branches of Engineering — First Year B.Tech

L : T : P : Credits = 0 : 1 : 3 : 2.5 | 12-Week Semester

Course Outline

This course introduces first-year engineering students to the practical use of Artificial Intelligence tools across problem-solving, data analysis, content creation, and coding.

Students gain hands-on experience with generative AI, prompt engineering, AI-assisted data visualization, and productivity/coding copilots through weekly lab practice.

The course emphasizes responsible and ethical AI use alongside technical competence, and serves as a foundation for AI-assisted work in higher-semester courses across all branches.

By the end, students will be able to independently select, apply, and evaluate AI tools for academic and engineering tasks.

Course Objectives (for the Teacher)

1. To familiarize students with the landscape of AI tools and their underlying working principles.
2. To develop competence in prompt engineering for effective interaction with generative AI systems.
3. To enable students to apply AI tools for data analysis, visualization, and interpretation.
4. To train students in using AI-assisted tools for coding, writing, and productivity tasks.
5. To instill awareness of ethical, legal, and societal considerations in AI tool usage.

Course Outcomes (Bloom's Taxonomy) — for Students

On successful completion of this course, students will be able to:

1. Recall the categories, capabilities, and limitations of major AI tools (text, image, code, data). (K1)
2. Explain the principles of prompt engineering and how generative AI models process instructions. (K2)
3. Apply AI tools to solve first-year engineering problems in analysis, writing, and

coding tasks. (K3)
4. Analyze AI-generated outputs for accuracy, bias, and reliability before use. (K4)
5. Evaluate the ethical, academic-integrity, and societal implications of AI tool use, justifying appropriate usage choices. (K5)

CO\ PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	M	–	–	–	H	L	L	–	–	–	L			
CO2	M	M	–	–	H	L	–	L	–	M	L			
CO3	H	H	M	M	H	L	L	–	L	M	M			
CO4	M	H	M	H	H	M	M	H	L	M	M			
CO5	L	M	M	M	M	H	H	H	M	H	M			

Unit-wise Syllabus

Unit 1: Introduction to AI and the AI Tools Landscape

Topics Covered:

1. What is AI, Machine Learning, and Generative AI — basic definitions and how they relate to one another.
2. Core categories of AI tools: conversational/text, code assistants, data-analysis tools, image-generation tools, and writing/productivity tools.
3. How large language models process and generate responses (conceptual, non-mathematical overview).
4. Criteria for selecting the right AI tool for a given task — accuracy needs, cost, data sensitivity, and output format.
5. Setting up accounts and workspaces; understanding free-tier versus paid-tier capabilities and limitations.

Widely Used AI Tools by Application (for classroom coverage):

Category	Example Tools	Typical Application
Conversational / General-Purpose AI	ChatGPT, Claude, Gemini, Perplexity	Writing, research, coding help, file/data analysis, cited web search
AI Coding Assistants	GitHub Copilot, Cursor, Claude Code	In-IDE code completion, AI-native editing, multi-file agentic coding
Data Analysis Tools	ChatGPT (Advanced Data Analysis), Microsoft	Upload a dataset, generate

	Copilot in Excel, Julius AI	statistics/charts, spreadsheet formulas via natural language
Image Generation	Midjourney, DALL·E 3, Stable Diffusion / Flux	Illustrations, concept art, design mockups
Writing & Productivity	Grammarly, Gamma, Fathom, Notion AI	Grammar/style checking, slide generation, meeting summaries, task automation
Audio, Video & Presentation (optional)	ElevenLabs, Runway/Sora, Synthesia	Text-to-speech, AI video generation, script-to-video presentations

Unit Outcome(s):

1. Identify the categories of AI tools available and select an appropriate tool for a given academic or engineering task.

Teacher Practical Examples (for classroom demonstration):

1. Live comparison of the same general engineering query (e.g., "explain the engineering design process") answered by two different AI chatbots, highlighting differences in response style and accuracy.
2. Walkthrough of an AI tool's interface — settings, conversation history, file upload, and model selection — using a common platform.

Student Exercise Problems (for practice):

1. Create accounts on two AI platforms and compare their responses to a simple engineering question (e.g., "explain the difference between analysis and design").
2. Prepare a one-page comparison chart of five AI tools by category and primary use case.
3. Write a short reflection identifying one task in your daily academic life where an AI tool could help.

Unit 2: Prompt Engineering and Generative AI Tools

Topics Covered:

1. Anatomy of a good prompt — context, clear instruction, desired format, and constraints.
2. Zero-shot, few-shot, and chain-of-thought prompting techniques and when to use each.

3. Role-based and context-setting prompts (e.g., asking the AI to respond as a reviewer, tutor, or domain expert).
4. Iterative prompt refinement — diagnosing a poor AI response and rewriting the prompt to fix it.
5. Using generative AI for brainstorming, summarizing, and drafting technical content.

Unit Outcome(s):

1. Construct clear, well-structured prompts using zero-shot, few-shot, and chain-of-thought techniques to obtain reliable AI output.

Teacher Practical Examples (for classroom demonstration):

1. Refine a vague prompt (e.g., "explain project scheduling") into a precise one step-by-step, showing output improvement at each stage.
2. Demonstrate few-shot prompting by providing examples to guide the AI toward a specific output format (e.g., a formatted comparison table).

Student Exercise Problems (for practice):

1. Write three versions of a prompt (basic, detailed, role-based) for the same task and compare outputs.
2. Use an AI tool to summarize a one-page technical article and verify the summary against the original.
3. Draft a chain-of-thought prompt to solve a multi-step numerical/logical problem (e.g., a unit-conversion or estimation problem) and check the reasoning.

Unit 3: AI Tools for Data Analysis and Visualization

Topics Covered:

1. AI-assisted spreadsheet and dataset analysis using natural-language commands.
2. Natural-language querying of datasets to extract summary statistics and insights.
3. AI tools for generating charts, graphs, and visual summaries from tabular data.
4. Interpreting AI-generated statistical summaries correctly and recognizing their limitations.
5. Identifying and correcting errors or misinterpretations in AI-generated data analysis.

Unit Outcome(s):

1. Use AI tools to clean, summarize, and visualize a dataset via natural-language commands, and identify errors in AI-generated analysis.

Teacher Practical Examples (for classroom demonstration):

1. Upload a small dataset (e.g., class attendance or survey data) to an AI tool and generate a summary-statistics report and chart using natural-language

commands.

2. Show an example where an AI tool misinterprets data (e.g., treats a numeric code as text), and demonstrate how to verify and correct it.

Student Exercise Problems (for practice):

1. Use an AI tool to clean and summarize a small sample dataset (e.g., class attendance or marks data).
2. Generate three different chart types from the same dataset using AI prompts and justify which is most appropriate.
3. Cross-check one AI-generated statistical claim manually and report any discrepancy found.

Unit 4: AI Tools for Coding, Writing, and Productivity

Topics Covered:

1. AI coding assistants — code generation, explanation, and debugging support for simple programs.
2. AI writing assistants for reports, emails, and technical documentation.
3. AI tools for scheduling, note-taking, and meeting/lecture summarization.
4. AI-assisted presentation and slide-outline generation from a topic prompt.
5. Integrating AI tools into an efficient day-to-day academic and engineering workflow.

Unit Outcome(s):

1. Apply AI coding assistants and writing tools to complete and refine a simple technical task.
2. Integrate multiple AI tools into a coherent workflow.

Teacher Practical Examples (for classroom demonstration):

1. Use an AI coding assistant to generate a simple program (e.g., a temperature converter), then intentionally introduce a bug and have the AI debug it.
2. Generate a presentation outline and speaker notes from a topic prompt using an AI writing/productivity tool.

Student Exercise Problems (for practice):

1. Use an AI coding tool to write and debug a simple program for a given problem statement.
2. Draft a formal email or short report using an AI writing assistant, then manually edit it for accuracy and tone.
3. Generate a slide outline for a 5-minute seminar topic using an AI tool and refine it.

Unit 5: Ethics, Reliability, and Emerging Trends

Topics Covered:

1. Bias and hallucination in AI outputs, and techniques for detecting and fact-checking them.
2. Academic integrity and appropriate citation of AI assistance in coursework and submissions.
3. Data privacy and security considerations, including obligations under India's Digital Personal Data Protection Act, 2023 (DPDP Act), when uploading personal or institutional data.
4. Copyright and intellectual property considerations in AI-generated text and image content.
5. Emerging trends — multimodal AI and AI agents — and their implications for future engineering practice.

Unit Outcome(s):

1. Evaluate AI-generated output for hallucination, bias, and copyright concerns, and apply ethical and data-privacy principles to AI tool use.

Teacher Practical Examples (for classroom demonstration):

1. Show a documented case of AI hallucination (a fabricated fact or citation) and demonstrate how to fact-check it.
2. Walk through an example of properly citing AI assistance in an academic submission per institutional guidelines.
3. Walk through a worked example of checking whether AI-generated text or image output raises a copyright/IP concern (e.g., close paraphrasing of a source).

Student Exercise Problems (for practice):

1. Identify and document a hallucination or bias instance produced by an AI tool on a factual query.
2. Draft an "AI usage declaration" for a hypothetical assignment, citing tools and extent of use appropriately.
3. Review the data-handling/privacy policy of one AI tool used in this course and identify one DPDP Act, 2023 compliance consideration relevant to student data.

Text Books

1. Marr, B. (2024). Generative AI in practice: 100+ amazing ways generative artificial intelligence is changing business and society. Wiley.
2. Mollick, E. (2024). Co-intelligence: Living and working with AI. Portfolio/Penguin.

Reference Books

1. Selwyn, N. (2022). Education and technology: Key issues and debates (3rd ed.). Bloomsbury.

2. Susskind, R., & Susskind, D. (2022). The future of the professions: How technology will transform the work of human experts (Updated ed.). Oxford University Press.
3. Russell, S., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.

Web Resources

1. Google. (n.d.). Google AI essentials. Grow with Google. https://grow.google/ai-essentials/
2. OpenAI. (n.d.). OpenAI documentation and guides. https://platform.openai.com/docs
3. Anthropic. (n.d.). Prompt engineering overview. Claude Docs. https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
4. UNESCO. (2023). Guidance for generative AI in education and research. https://unesdoc.unesco.org/
5. AICTE. (2025). AICTE inputs on Artificial Intelligence — Year of AI initiative. https://www.aicte.gov.in/downloads/initiatives/AICTE%20inputs%20on%20Artificial%20Intelligence%20-compre hensive.pdf
6. Ministry of Electronics and Information Technology, Government of India. (2023). Digital Personal Data Protection Act, 2023. https://www.meity.gov.in/content/digital-personal-data-protection-act-2023

Review Questions

1. Explain the difference between AI, Machine Learning, and Generative AI with a suitable example.
2. Describe the anatomy of a good prompt and illustrate zero-shot versus few-shot prompting with an example.
3. Discuss two ways AI tools support data analysis and visualization, with an example of each.
4. What is meant by hallucination in AI-generated output? Explain how it can be detected and verified.
5. Discuss the ethical, academic-integrity, and data-privacy considerations relevant to using AI tools for coursework, referencing the DPDP Act, 2023.

Practical Applications

1. Using AI tools to accelerate literature review and topic understanding for mini-

projects and seminars.

2. Applying AI-assisted data analysis to quickly clean, summarize, and visualize experimental or survey data.
3. Using AI coding assistants to write, explain, and debug simple programs across engineering assignments.
4. Leveraging AI writing assistants to draft clear reports, emails, and presentations for academic and professional communication.
5. Applying responsible AI-usage practices — verification, citation, and privacy awareness — in everyday academic work.